

Mini-Batch Gradient Descent

Contents

1. Overview	2
2. A family that contains the other two	2
3. How it works	2
4. Comparing the three paths	4
5. Choosing the batch size	4
6. Mini-batch in scikit-learn	5
7. Summary	7
8. Key terms	7

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Gradient descent ([Note 57](#)).
- **Compare with:** Stochastic gradient descent ([Note 59](#)).

1. Overview

Key point: Mini-batch gradient descent updates the coefficients after each small group of rows. It sits between batch (stable, slow) and stochastic (fast, noisy) gradient descent, and is the version most used in practice.

The three types of gradient descent differ only in how often they update the coefficients:

- **Batch:** once per epoch, after looking at all n rows.
- **Stochastic:** n times per epoch, after each single row.
- **Mini-batch:** once after each **batch**: a small group of rows, typically 16 to 256.

With 1,000 rows and a batch size of 100, the data splits into 10 batches, so there are 10 updates per epoch. A batch size of 10 would give 100 updates per epoch.

2. A family that contains the other two

Key point: Batch size n is batch gradient descent; batch size 1 is stochastic gradient descent; anything between is mini-batch.

The **batch size** is a hyperparameter, and the other two types are its extreme values:

Batch size	Updates per epoch	Name
n (all rows)	1	batch gradient descent
1	n	stochastic gradient descent
between 1 and n	$n / \text{batch size}$	mini-batch gradient descent

So mini-batch is the general form, and the batch size tunes how it behaves between the two extremes.

3. How it works

Key point: Each epoch: shuffle the rows, cut them into batches, and for each batch compute the derivatives from its rows only and update.

1. Start with any coefficients, for example $\beta_0 = 0$ and every $\beta_j = 1$.
2. For each epoch:

- shuffle the rows into a random order;
- cut them into consecutive batches of the chosen size (the last batch may be smaller);
- for each batch, compute the predictions, the errors and the derivatives using only its rows, and update.

The derivatives are the batch ones, averaged over the rows B of the current batch instead of all n rows:

$$\frac{\partial L}{\partial \beta_j} = -\frac{2}{|B|} \sum_{i \in B} (y_i - \hat{y}_i) x_{ij}$$

where $|B|$ is the number of rows in the batch.

Python: Mini-batch gradient descent from scratch.

```
class MBGDRegressor:
    def __init__(self, batch_size, learning_rate=0.01,
                 epochs=100, seed=0):
        self.bs, self.lr, self.epochs = (batch_size,
                                           learning_rate, epochs)
        self.rng = np.random.default_rng(seed)

    def fit(self, X, y):
        n = X.shape[0]
        self.intercept_ = 0.0
        self.coef_ = np.ones(X.shape[1])
        for _ in range(self.epochs):
            order = self.rng.permutation(n)           # shuffle
            for start in range(0, n, self.bs):
                j = order[start:start + self.bs]      # one batch
                err = y[j] - (X[j] @ self.coef_ + self.intercept_)
                self.intercept_ -= self.lr * (-2 * err.mean())
                self.coef_ -= self.lr * (-2 * X[j].T @ err / len(j))
        return self
```

The derivative code is the same vectorised code as batch gradient descent, applied to the rows $X[j]$ of one batch.

Extra: Some implementations draw each batch at random from all rows, so rows can repeat within an epoch. Shuffling once and slicing, as above, uses every row exactly once per epoch and is the usual choice.

4. Comparing the three paths

Key point: In three epochs, batch barely moves, stochastic arrives but zigzags, and mini-batch arrives on a smoother path.

Figure 1 runs all three for 3 epochs on the 100-point example, with the same learning rate of 0.05.

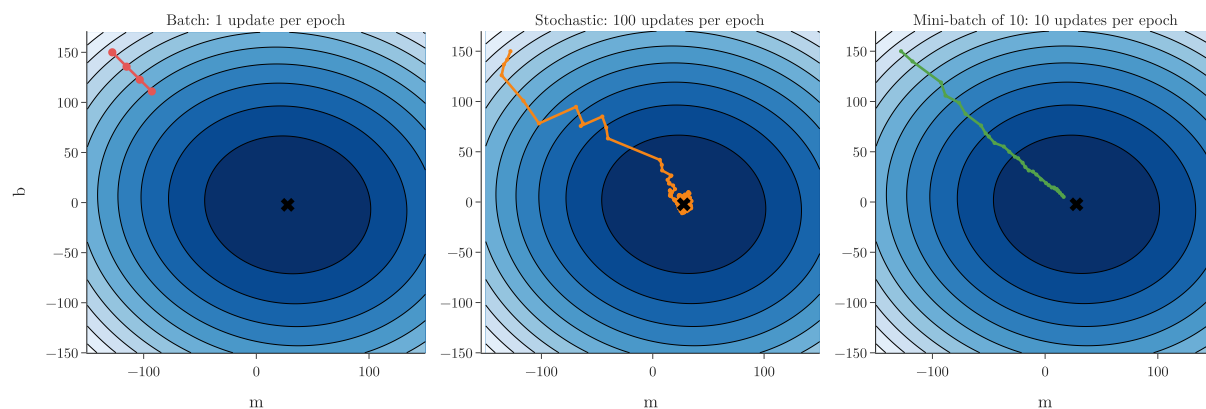


Figure 1: Batch, stochastic and mini-batch gradient descent over 3 epochs

- **Batch** (3 updates in total) has only just started.
- **Stochastic** (300 updates) reaches the minimum, then jumps around it.
- **Mini-batch of 10** (30 updates) follows a much smoother path and is almost there.

Averaging the derivative over 10 rows cancels much of the noise of a single row, while still updating 10 times as often as batch.

5. Choosing the batch size

Key point: Smaller batches mean more updates per epoch but noisier ones; larger batches mean smoother but fewer updates. Small batches such as 8 to 32 usually work well.

Figure 2 trains on the diabetes data with learning rate 0.1 for 100 epochs and four batch sizes.

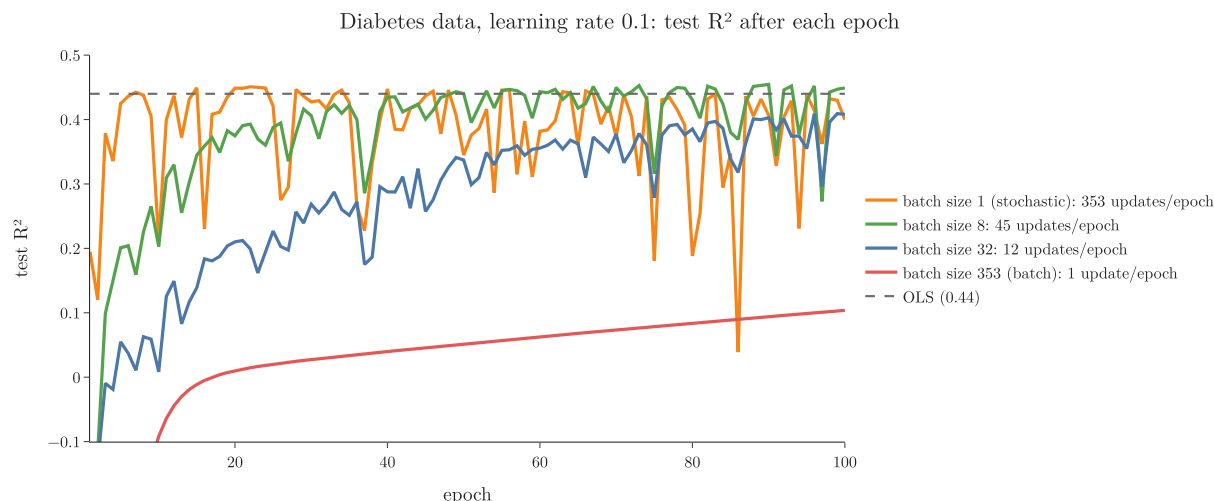


Figure 2: Test R^2 per epoch for four batch sizes

Batch size	Updates per epoch	Test R^2 after 100 epochs
1 (stochastic)	353	0.40
8	45	0.45
32	12	0.41
353 (batch)	1	0.10

- With batch size 1, accuracy climbs fastest at first but then jumps wildly from epoch to epoch.
- With batch size 8, it gets close to OLS (0.44) early and stays there more steadily.
- With batch size 32, it is smooth but slower, with fewer updates per epoch.
- Full batch has made only 100 updates in total and is far from done.

Like the learning rate, the batch size is tuned by trying values. The two interact: a larger batch gives a less noisy derivative, so it can often take a larger learning rate.

Extra: Batch sizes are often powers of two (16, 32, 64, 128) because computer hardware, especially GPUs in deep learning, processes arrays of these sizes efficiently. Mini-batches also need only one batch in memory at a time, which is how models are trained on datasets far larger than memory.

6. Mini-batch in scikit-learn

Key point: SGDRegressor has no batch size option, but its `partial_fit` method trains on one batch at a time, so a short loop gives mini-batch training.

SGDRegressor (from the previous Note) always updates one row at a time inside `fit`. Its `partial_fit` method, however, does one pass over whatever rows it is given, keeping the coefficients learned so far. Feeding it one batch at a time gives mini-batch-style training:

Python: Mini-batch training with `partial_fit`.

```
from sklearn.linear_model import SGDRegressor

sgd = SGDRegressor(learning_rate="constant", eta0=0.2,
                    random_state=0)
rng = np.random.default_rng(0)
for epoch in range(100):
    order = rng.permutation(n)
    for start in range(0, n, 35):
        j = order[start:start + 35]
        sgd.partial_fit(X_train[j], y_train[j])
sgd.score(X_test, y_test)      #  $R^2$  0.45
```

`partial_fit` is also how scikit-learn models learn from data that arrives over time, the online learning of Note 5.

Extra: Inside `partial_fit`, `SGDRegressor` still updates once per row of the batch rather than once per batch. True mini-batch updates (one update from the batch's average derivative) are standard in deep-learning libraries, where the batch size is a basic setting.

7. Summary

	Batch	Mini-batch	Stochastic
Rows per update	n	a small batch	1
Updates per epoch	1	n / batch size	n
Path	smooth, slow	fairly smooth, fast	noisy, fast
Memory	whole dataset	one batch	one row
Typical use	small data	most practice, deep learning	very large or streaming data

- Mini-batch gradient descent includes the other two as batch sizes n and 1.
- Each epoch: shuffle, cut into batches, update once per batch with the batch's average derivative.
- Batch size and learning rate are tuned together; small batches of 8 to 32 worked best here.
- In scikit-learn, `partial_fit` on successive batches gives mini-batch-style training.

8. Key terms

Term	Meaning
Mini-batch gradient descent	Gradient descent that updates after each small group of rows
Batch (mini-batch)	A small group of training rows used for one update
Batch size	The number of rows in each batch; a hyperparameter
Shuffling	Putting the rows in a new random order before each epoch
<code>partial_fit</code>	A scikit-learn method that continues training on new rows, keeping what was learned so far